

CollabSpace Frontend Full Explanation Guide

This guide explains the whole frontend codebase file by file, with architecture flow, syntax meaning, and visual understanding.

Backend is intentionally excluded based on your request.

1) Big Picture: What This Frontend Does

This frontend is a React + TypeScript + Vite SPA for multi-tenant project collaboration:

- Authentication (login/register)
- Organization management
- Member/role management
- Project management
- Kanban board (task drag/drop by status)
- Task detail with comments, activity, assignment history
- Tag management
- Light/dark theme support

Runtime visual flow

```text

Browser loads index.html

- > src/main.tsx mounts React app
- > Providers wrap app:
  - ColorModeProvider -> AuthProvider -> ThemedApp
  - > ConfigProvider (Ant theme by mode)
  - > RouterProvider (all routes)
    - > ProtectedRoute gates private routes
    - > AppShell layout
    - > Page components

```

Data flow visual

```text

UI action (button/form)

- > Page component handler
- > Service call (axios)
  - > Backend API
  - > ApiResponse<T>
  - <- Result
- > setState/useEffect refresh
- > UI rerender

```

2) Tech Stack and Why It Matters

- React 19 + TypeScript: component logic + type safety
- Vite: fast dev server and build
- React Router v7: client-side route tree

- Ant Design: ready-made UI components
- Tailwind v4: utility class styling (including 'important' suffix style like 'text-sm!')
- Axios: HTTP client with interceptors
- ESLint + hooks/refresh rules: code quality and safe Fast Refresh

3) Root-Level Files (Frontend Root)

'package.json'

- Defines scripts: 'dev', 'build', 'lint', 'preview'
- Dependencies include React, Router, AntD, Axios, Tailwind Vite plugin
- This is where build and lint behavior starts.

'package-lock.json'

- Auto-generated dependency lock.
- Ensures exact package versions for consistent install.

'vite.config.ts'

- Registers plugins:
 - '@vitejs/plugin-react'
 - '@tailwindcss/vite'

'eslint.config.js'

- Enables:
 - JS recommended rules
 - TypeScript ESLint recommended rules
 - React hooks rules
 - React refresh (Vite) rules
- Important for warnings you recently fixed.

'tsconfig.json', 'tsconfig.app.json', 'tsconfig.node.json'

- Project references and compiler options.
- App config includes DOM libs and JSX mode.
- Node config covers Vite config typing.

'index.html'

- Single root element '<div id="root"></div>'
- Boot script points to '/src/main.tsx'.

'.gitignore'

- Standard ignore behavior for node, build artifacts, etc.

'README.md'

- Vite template README (not project-specific docs).

4) 'src/' Top-Level Files

'src/main.tsx'

Main entry point:

- Gets theme mode from 'useColorMode()'
- Applies Ant Design theme through 'ConfigProvider'

- Injects router through 'RouterProvider'
- Wraps with 'ColorModeProvider' and 'AuthProvider'
- Uses singleton root pattern to avoid duplicate 'createRoot' warnings in HMR:

```

""tsx
const root = windowWithRoot.__collabSpaceRoot__ ?? createRoot(container);
windowWithRoot.__collabSpaceRoot__ = root;
""

```

'src/App.tsx'

- Minimal route shell returning '<Outlet />'
- Used as top route parent.

'src/index.css'

Global styling system:

- Tailwind import: '@import "tailwindcss";'
- CSS variables for light/dark theme ('--surface', '--ink', '--primary', etc.)
- Sidebar/topbar card styles
- Utility classes for repeated layout blocks ('compact-list', 'compact-metric', etc.)
- This file defines most of the app visual identity.

5) App Setup Layer ('src/app')

'src/app/router.tsx'

Defines full route tree:

- Public: '/login', '/register'
- Protected (inside 'ProtectedRoute' + 'AppShell'):
 - '/dashboard'
 - '/organizations', '/organizations/:organizationId', '/organizations/:organizationId/members'
 - '/projects', '/projects/:projectId', '/projects/:projectId/board'
 - '/board' (redirect page)
 - '/tasks/:taskId'
 - '/tags'
 - '/settings'

'src/app/theme.ts'

Function 'getAppTheme(mode)' returns Ant theme config:

- Switches 'theme.darkAlgorithm' vs default
- Custom tokens: colors, fonts, radius, shadows
- Component-level tokens for Button/Card/Input

6) Hooks Layer ('src/hooks')

'src/hooks/use-color-mode.tsx'

Color mode context/provider:

- Mode type: 'light' | 'dark'
- Persists mode in localStorage key 'collabSpace_color_mode'
- Applies mode to 'document.documentElement.dataset.theme'
- Exposes:
 - 'mode'

- 'toggleMode()'
- 'setMode()'

'src/hooks/auth-context.ts'

- Holds auth context type and context instance only.
- Split intentionally for Fast Refresh lint compatibility.

'src/hooks/auth-provider.tsx'

Owns auth state and business flow:

- State:
 - 'user'
 - 'loading'
 - 'activeOrganizationId'
- Bootstraps '/auth/me' only once via 'hasBootstrappedRef'
- 'login()' stores token + user + active org
- 'register()' calls register API (no auto-login now)
- 'logout()' clears token and active org
- 'refreshMe()' refetches profile

'src/hooks/use-auth.ts'

- Simple custom hook:
 - reads AuthContext
 - throws if used outside 'AuthProvider'

7) Shared Utility Layer

'src/lib/storage.ts'

LocalStorage abstraction:

- token key: 'collabspace_token'
- active org key: 'collabspace_active_org'
- centralized get/set/clear methods

'src/types/api.ts'

- 'ApiResponse<T>' structure from backend
- union types for 'OrganizationRole', 'TaskStatus'

'src/types/entities.ts'

Defines app entity contracts:

- 'User', 'Organization', 'Project', 'TagEntity', 'TaskEntity', etc.
- 'TaskEntity' includes 'createdBy', 'assignedUser', tags, comment fields
- These types power service signatures and page state typing.

8) API Service Layer ('src/services')

Common pattern

All services follow:

```

"""ts
const { data } = await apiClient.get<ApiResponse<SomeType>>(url);
return data.Result;
"""

```

'src/services/api.ts'

Axios client + interceptor:

- Base URL from 'VITE_API_BASE_URL' fallback 'http://localhost:5000/api'
- Adds 'Authorization: Bearer <token>' if token exists
- Adds 'x-organization-id' header if active org exists
- 'getErrorMessage()' normalizes Axios/non-Axios error text

'src/services/auth.ts'

- 'login', 'register', 'me'
- normalizes email before send
- note: register expects 'AuthUser' response, not token payload

'src/services/organizations.ts'

- organization CRUD + members CRUD
- includes role update and remove member

'src/services/projects.ts'

- list/create/get/update/remove project

'src/services/tags.ts'

- list/create/update/remove tag

'src/services/tasks.ts'

- list/board/create/get/update/archive task
- comment endpoints
- activity and assignment-history endpoints

9) Reusable UI Components ('src/components')

'src/components/common/feedback.tsx'

- 'PageLoader', 'CardLoader', 'EmptyState', 'ErrorState'
- Provides loading/error empty UX consistency

'src/components/common/ui.tsx'

Reusable domain UI:

- 'SectionHeader'
- 'StatCard'
- 'StatusBadge' (Task status to Tag color)
- 'RoleBadge' (Role to Tag color)
- 'IconActionButton' (tooltip icon button)
- 'ProjectCard', 'TaskPreviewCard'

This prevents repeating identical card/badge/header logic in every page.

'src/components/routing/protected-route.tsx'

Auth gate:

- If loading: loader
- If no user: redirect '/login'
- Else: render nested routes via '<Outlet />'

'src/components/layout/app-shell.tsx'

Main authenticated layout:

- Left sidebar with navigation and org selector
- Top bar with workspace title + collapse toggle
- Theme toggle and logout controls
- Uses 'useAuth' and 'useColorMode'
- Fetches project list to decide if board menu should be enabled

10) Feature Entry Files

'src/features/pages.tsx'

- Explicit re-export list for all page components.
- Router imports from this single entry.

'src/features/pages/index.ts'

- Internal pages folder barrel, also explicit exports.

'src/features/pages/shared.ts'

Shared constants/helpers:

- color presets
- role and status options
- permission helpers:
 - 'canManageMembers'
 - 'canManageProjects'
 - 'canManageTags'
 - 'canEditOrganization'
 - 'canCreateOrganizations'
- normalization helper for email
- options mappers for members/tags
- 'useSelectedOrganization()' reads auth context and returns current org and membership

11) Page Files ('src/features/pages/*')

Below is route-order flow from first login to deep task work.

11.1 'login-page.tsx' ('/login')

- Form for email/password
- Calls 'login()'
- On success: message + navigate '/dashboard'
- If already authenticated, auto-redirects to dashboard in 'useEffect'

11.2 'register-page.tsx' ('/register')

- Form for first admin + organization creation
- Calls 'register()'
- On success: redirects to '/login' (intentional, no auto-login)

11.3 'dashboard-page.tsx' ('/dashboard')

- Loads active organization details and projects
- Computes:
 - project counts
 - per-project task count

- recently updated tasks
- Shows top-level workspace summary widgets

11.4 'organizations-page.tsx' ('/organizations')

- Shows org cards from memberships
- Create organization modal (permission-guarded)
- Buttons to open org detail and member list

11.5 'organization-detail-page.tsx' ('/organizations/:organizationId')

- Loads one organization
- Displays stats: members/projects/tags
- Shows member preview and project/tag preview
- Edit organization modal (admin only)

11.6 'members-page.tsx' ('/organizations/:organizationId/members')

- Member list + role badges
- Register member modal
- Update member role modal
- Remove member action with centered 'modal.confirm'

11.7 'projects-page.tsx' ('/projects')

- Project portfolio cards
- Create/edit project modal
- Archive project confirmation modal
- Quick actions:
 - open details
 - open board
 - edit
 - archive

11.8 'project-detail-page.tsx' ('/projects/:projectId')

- Fetches project + tasks
- Splits tasks into active/archived via 'useMemo'
- Tabs for active and archived lists

11.9 'board-redirect-page.tsx' ('/board')

- Finds first project in active org
- Redirects to that project board
- If none, shows empty state

11.10 'board-page.tsx' ('/projects/:projectId/board')

- Kanban board with 3 columns from 'statusOptions'
- Loads project, board data, members, tags
- Create task modal
- Drag-and-drop task movement:
 - on drop, calls 'taskService.update(taskId, {status})'
 - optimistically updates local board state
 - reload fallback on failure

11.11 'task-detail-page.tsx' ('/tasks/:taskId')

- Full task detail view
- 3-dot menu:
 - Edit task (modal form)
 - Delete task (archives with confirm modal)

- Displays:
 - status + active/archived badge
 - description
 - assignee
 - created by
 - created/updated timestamp
 - tags
- Tabs:
 - comments (add + list)
 - activity history
 - assignment history

11.12 'tags-page.tsx' ('/tags')

- List and manage tags
- Create/edit modal with accent color picker
- Delete confirmation via centered modal

11.13 'settings-page.tsx' ('/settings')

- Profile and access summary
- simple refresh profile action

12) Route and Permission Summary (Visual)

```text

Public:

/login

/register

Protected:

/dashboard

/organizations

/organizations/:organizationId

/organizations/:organizationId/members

/projects

/projects/:projectId

/projects/:projectId/board

/board

/tasks/:taskId

/tags

/settings

```

Permission decisions are UI-side helper checks in 'pages/shared.ts':

- Admin:
 - create org
 - manage members
 - edit org
- Admin or Manager:
 - manage projects
 - manage tags

13) Important Syntax Patterns You Should Explain in GD

1. React context provider + custom hook guard:

```
""ts
const context = useContext(AuthContext);
if (!context) throw new Error(...);
""
```

2. API typing with generics:

```
""ts
ApiClient.get<ApiResponse<Project[]>>(...)
""
```

3. Conditional route guard:

```
""tsx
if (!user) return <Navigate to="/login" replace />;
""
```

4. useEffect bootstrap pattern:

```
""ts
const hasBootstrappedRef = useRef(false);
""
```

5. Memoized loaders with 'useCallback' for lint-safe dependencies:

```
""ts
const loadData = useCallback(async () => {...}, [deps]);
useEffect(() => { void loadData(); }, [loadData]);
""
```

6. AntD modal confirm for destructive actions:

```
""ts
modal.confirm({ title, onOk: async () => await removeX() });
""
```

7. Tailwind v4 important suffix style:

```
""tsx
className="text-slate-500!"
""
```

14) End-to-End User Journey (Mentor-Friendly Story)

1. User opens app -> 'main.tsx' mounts providers + router.
2. AuthProvider checks token:
 - if token valid -> fetch '/auth/me' and open private area
 - else -> redirect to login
3. After login:
 - token saved
 - active organization selected
 - AppShell appears
4. User manages organizations/projects/members/tags
5. User opens board:

- sees tasks grouped by status
 - drags task to change status
6. User opens task detail:
- reviews full task context
 - edits through modal
 - comments and checks activity/history

15) Frontend GD Preparation Checklist

- Explain why 'apiClient' interceptor centralizes auth/org headers.
- Explain why context split ('auth-context', 'auth-provider', 'use-auth') helps Fast Refresh lint.
- Explain route nesting ('ProtectedRoute' + 'AppShell') and why it avoids duplication.
- Explain why shared UI components improve maintainability.
- Explain board drag/drop state update path and fallback reload.
- Explain difference between UI permissions (client) and real backend authorization (server).
- Explain theme flow:
 - mode in localStorage
 - CSS vars on root
 - Ant theme tokens from mode

16) File Coverage Checklist (Nothing Missed)

Covered:

- Root frontend config files: yes
- All 'src' files listed by 'rg --files': yes
- All route pages: yes
- All hooks/services/types/components: yes
- Shared and barrel files: yes

This is the complete frontend explanation scope.